

 NYU

LLM-Generated Reward Shaping for Reinforcement Learning in Super Mario Bros

Can a large language model replace a human reward engineer through iterative reward design for a PPO-based Mario agent?

PRESENTED BY:

Mustafa Poonawala (msp9471)

Siya Koppikar (sk11806)

Kund Nilesh Meghani (km6579)

Research Question: Can an LLM design & iteratively improve a reward function that trains a PPO agent to play Super Mario as well as a hand-crafted reward?

WHY RL?

Reward design is one of the hardest parts of RL. We study whether an LLM can automate this step.

Mario has sequential decision-making, sparse/structured rewards, and long-horizon planning, a natural testbed for reward shaping.

FORMAL RL FORMULATION

Env: gym-super-mario-bros

Agent: PPO via Stable-Baselines3

State: Mario game frames (pixels)

Action: Discrete movement/action set

Reward: Default OR LLM-generated

WHY THIS PROBLEM MATTERS

Reward Engineering is Hard

Requires substantial human intuition and trial-and-error which is a major bottleneck in RL.

LLMs Could Automate It

If LLMs can design rewards, RL becomes faster and more accessible to practitioners.

Mario is an Ideal Benchmark

Clean, well-known environment, that is, behavior is easy to visualize and evaluate.

Related Work

- EUREKA (NVIDIA 2023): LLMs generate reward functions for robotics which is our direct inspiration.
- Large-Scale Study of Curiosity-Driven Learning (Burda et al., 2018): showed PPO-based agents can learn in Super Mario Bros, providing a precedent for PPO in this environment.
- Text2Reward: Reward Shaping with Language Models for Reinforcement Learning Xie et al. - ICLR 2024

Our Original Contribution

- Apply LLM reward design to Super Mario Bros
- Iterative loop: LLM proposes → PPO trains → curves fed back → LLM revises
- Compare: baseline vs LLM v1 vs refined LLM v2

Method, Design Choices & Feasibility

Pipeline

- 1 Describe Mario environment & goal to the LLM
- 2 LLM writes a Python reward function
- 3 Train PPO agent for fixed timestep budget
- 4 Collect training curves and evaluation metrics
- 5 Feed results back to the LLM
- 6 LLM generates revised reward → retrain
- 7 Compare all 3 runs side-by-side

Tech Stack & Justification

ENVIRONMENT

gym-super-mario-bros

pip installable, reproducible

ALGORITHM

PPO (Stable-Baselines3)

stable, good for discrete control

LLM

Gemini Flash

free tier, ~10–20 calls total

COMPUTE

Local System

LANGUAGE

Python 3.8 / 3.9

pinned for gym compatibility

Experimental Runs & Metrics

Run 1	Default Gym Reward Baseline: Default environment reward under the same timestep budget	Avg x-pos / run
Run 2	LLM-Generated Reward v1 Core Contribution	Completion Rate
Run 3	LLM-Refined Reward v2 Iterative Improvement	Sample efficiency

All Runs Evaluated On:

- Level completion rate
- Average forward progress per episode
- Score improvement over time
- Sample efficiency (learning speed)
- Final performance under a fixed compute budget

Robustness:

- Tested with multiple random seeds
- Measured consistency across runs



Project Milestones

- Week 1**
Environment setup + baseline PPO training started
- Week 2**
LLM integration + first generated reward function
- Week 3**
All 3 training runs completed
- Week 4**
Evaluation plots, conclusions, final slides

Expected Outcome:

We expect the LLM-designed reward to improve learning efficiency. Even if improvements were limited, the results still help highlight where LLM-guided reward design works and where it doesn't, giving useful insight into its practical limits.